Protocol Audit Report

Prepared by: Srishti
Date: March 23, 2026
Version: 1.0
Status: `CONFIDENTIAL`

Table of Contents

- Protocol Audit Report
  - Table of Contents
  - Protocol Summary
  - Disclaimer
  - Risk Classification
  - Audit Details
    - Scope
    - Roles
  - Executive Summary
    - Issues Found
  - Findings
  - High
    - [H-1] Password stored on-chain is publicly readable
      - Description
      - Impact
      - Proof of Concept
      - Recommended Mitigation
    - [H-2] `setPassword` has no access controls — any address can overwrite the password
      - Description
      - Impact
      - Proof of Concept
      - Recommended Mitigation
  - Informational
    - [I-1] Incorrect natspec parameter on `getPassword`

- Description
- Impact
- Recommended Mitigation

Protocol Summary

PasswordStore is a smart contract protocol for the secure storage and retrieval of a single user's password. The protocol is explicitly designed for single-user operation only. According to the specification, only the designated owner should be able to set and access the stored password.

Disclaimer

> **Independent Security Review**
> Every effort was made to identify vulnerabilities within the allotted time. However, this report does not constitute an endorsement of the underlying business or product. The audit was time-boxed and focused solely on the security aspects of the Solidity implementation. The auditor assumes no responsibility for the findings presented herein.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	H	H/M	M
Likelihood: Medium	H/M	M	M/L
Likelihood: Low	M	M/L	L

Severity is determined using the CodeHawks severity matrix.

Audit Details

Commit Hash

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
./src/
PasswordStore.sol
```

Roles

- **Owner** — May set and retrieve the password.
- **Outsiders** — Must not be able to set or retrieve the password under any circumstances.

Executive Summary

The PasswordStore contract was reviewed for security vulnerabilities. The audit revealed two critical High-severity findings and one Informational finding. The High-severity issues collectively render the protocol's core security guarantees entirely broken in its current state — the password is exposed to the public regardless of access controls, and access controls themselves are absent.

> **Deployment to any live network is not recommended without addressing both High findings.**

Issues Found

Severity	Number of Issues
High	2
Medium	0
Low	0
Informational	1
Total	**3**

Findings

High

[H-1] Password stored on-chain is publicly readable

Severity: `HIGH`
Impact: HIGH · **Likelihood:** HIGH

Description All data stored on the Ethereum blockchain is publicly visible — the EVM storage model does not enforce privacy. The `PasswordStore::s_password` variable is marked **`private`** in Solidity, but this only prevents other contracts from reading it through the ABI; it does not prevent direct low-level storage reads.

Anyone can query storage slot `1` of the deployed contract using standard tools, bypassing the `PasswordStore::getPassword` access control entirely.

Impact

> Any actor can read the stored password directly from the blockchain without interacting with the contract's interface. The confidentiality guarantee of the protocol is completely broken.

Proof of Concept **1. Start a local Anvil chain:**

```
make anvil
```

2. Deploy the contract:

```
make deploy
```

3. Read storage slot 1 directly (`s_password` is stored at slot 1):

```
cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

Output:

```
0x6d7950617373776f72640000000000000000000000000000000000000000000014
```

4. Decode the hex to a human-readable string:

```
cast parse-bytes32-string 0
    x6d7950617373776f72640000000000000000000000000000000000000000000014
```

Output:

```
myPassword
```

Recommended Mitigation The contract architecture must be reconsidered. The blockchain is not a suitable storage medium for plaintext secrets.

- Encrypt the password **off-chain** using a key known only to the owner before storing it on-chain.
- Store only the **ciphertext** on-chain. The plaintext is never transmitted or stored on the blockchain.
- Remove the `getPassword` view function to prevent accidental on-chain exposure of the decryption key in a future transaction.
- Consider whether on-chain storage is needed at all, or whether an off-chain encrypted vault is more appropriate.

[H-2] `setPassword` has no access controls — any address can overwrite the password

Severity: `HIGH`
Impact: HIGH · **Likelihood:** HIGH

Description The `setPassword` function is declared `external` with no owner check. The natspec explicitly states that only the owner should be able to set a new password, but the implementation contains no such restriction. Any externally owned account or contract can call `setPassword` and replace the stored password.

```
function setPassword(string memory newPassword) external {
    // @audit - no access control
    s_password = newPassword;
    emit SetNewPassword();
}
```

Impact

> Any caller can set an arbitrary password, permanently locking out the legitimate owner and rendering the protocol's ownership model meaningless.

Proof of Concept Add the following fuzz test to `PasswordStore.t.sol`:

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.assume(randomAddress != owner);
    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

The test passes for any non-owner address, confirming the vulnerability.

Recommended Mitigation Add an owner-only guard at the top of `setPassword`:

```
function setPassword(string memory newPassword) external {
    if (msg.sender != s_owner) {
        revert PasswordStore_NotOwner();
    }
    s_password = newPassword;
    emit SetNewPassword();
}
```

Informational

[I-1] Incorrect natspec parameter on `getPassword`

Severity: `INFORMATIONAL`
Impact: NONE · **Likelihood:** HIGH

Description The NatSpec comment for `getPassword` documents a `@param newPassword` that does not exist. The function takes no parameters, making the documentation misleading for developers and automated documentation tools.

```
/*
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set.  // <-- incorrect
 */
function getPassword() external view returns (string memory)
```

Impact No runtime impact. The incorrect natspec may mislead developers reading the contract and will produce warnings or errors in documentation-generation tooling such as `solidity-docgen`.

Recommended Mitigation Remove the erroneous `@param` line:

```
- * @param newPassword The new password to set.
```

Report generated by Srishti · March 23, 2026